



IMAGE SEGMENTATION USING U-NET

Dataset: Indian Pines Hyperspectral Dataset

Presented by:

Vishva D Patel IU2441231791

Jenish Joshi IU2441231784

INTRODUCTION

- Image segmentation is a computer vision task in which an image is divided into meaningful regions by assigning a class label to each pixel.
- Unlike image classification, which assigns one label to an entire image, segmentation provides pixel-level predictions.
- Applications:- Remote sensing , Medical image analysis , Autonomous vehicles , Agriculture , Land-cover mapping , Object recognition , Satellite image analysis

INDIAN PINES HYPERSPECTRAL DATASET

- The Indian Pines dataset is a hyperspectral image dataset used to study and classify different regions or materials present in an image.
- Unlike a normal RGB image, which has only 3 color channels, a hyperspectral image contains information from many spectral bands.

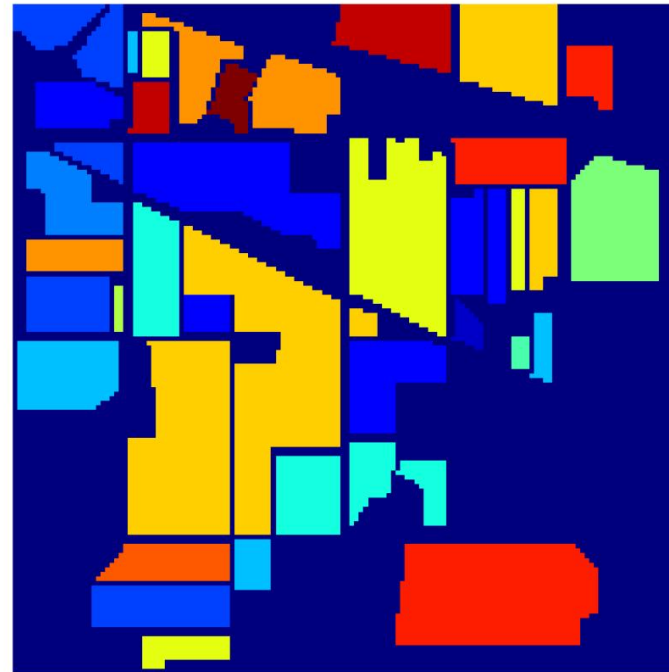
INDIAN PINES HYPERSPECTRAL DATASET

1 Original Indian Pines Dataset

Original Indian Pines Image



Original Ground Truth Mask



Deploy ⋮

⌕

OUR DATASET

- The image used in this project has:

Image size: $145 \times 145 \times 200$

$145 \times 145 \rightarrow$ height and width of the image

$200 \rightarrow$ number of spectral bands

- The dataset also contains a ground-truth mask of size: 145×145
- This mask tells us the correct class of each pixel and is used as the reference while training the model.
- The segmentation model is designed to identify 17 different classes in the image.

OUR DATASET

Dataset Information

Image Height

145

Image Width

145

Spectral Bands

200

Classes

17

Original image shape: (145, 145, 200)

Original mask shape: (145, 145)

PROBLEM STATEMENT

- A hyperspectral image contains a large amount of information for every pixel. The challenge is to automatically determine what class each pixel belongs to.
- Manually identifying every pixel in a large image is:
 - Time-consuming
 - Difficult to perform consistently
 - Dependent on human expertise
 - Not practical for large amounts of image data
- The goal of this project is to develop a deep-learning model that can automatically classify every pixel of the Indian Pines image into one of the 17 classes.

UNDERSTANDING IMAGE SEGMENTATION

- Image segmentation means dividing an image into meaningful regions and assigning a class to each pixel.
- For our project, the model looks at the image and determines which of the 17 classes each pixel belongs to.

WHY DO WE NEED DEEP LEARNING

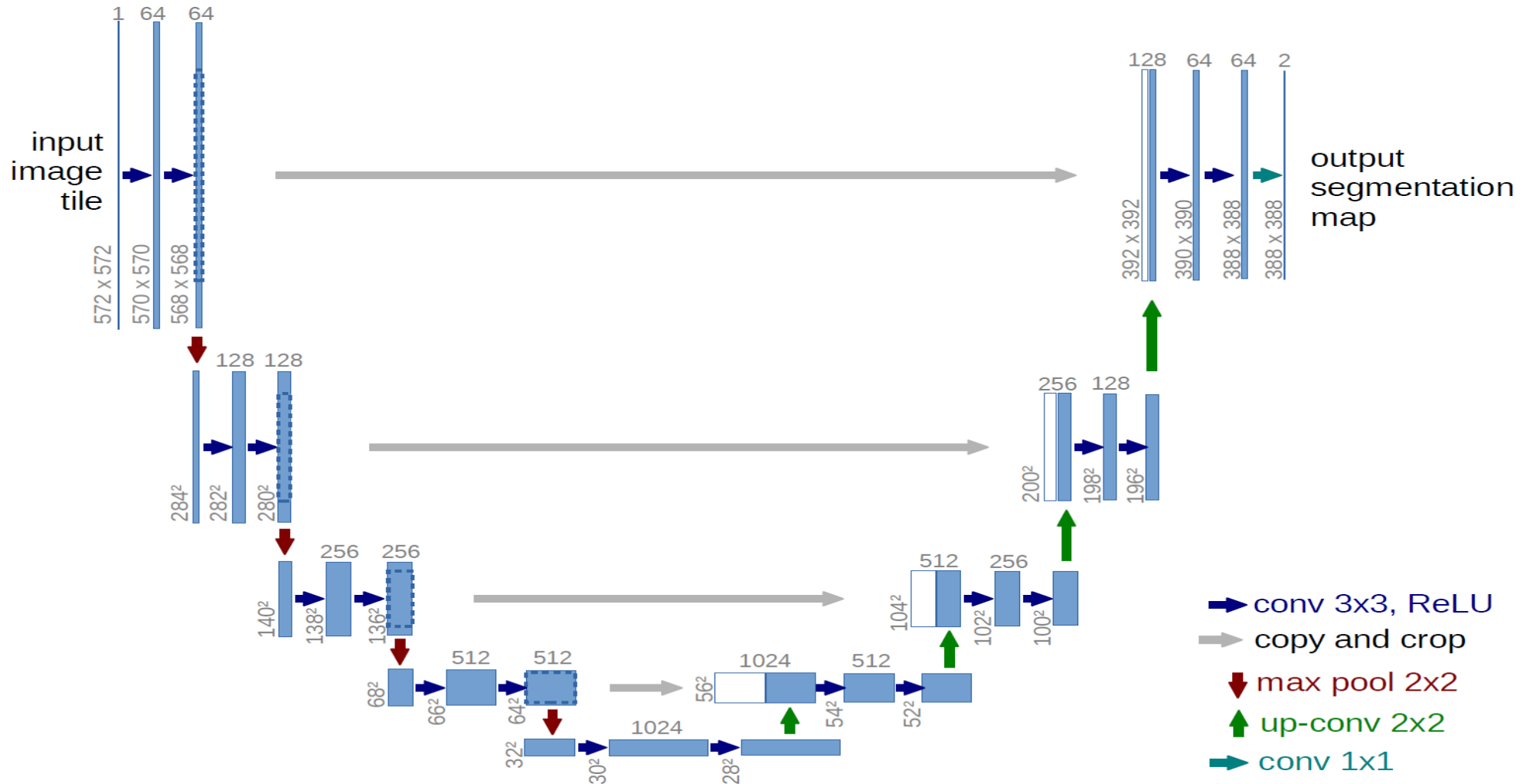
- The Indian Pines image contains a large amount of information because each pixel has information from 200 spectral bands.
- Creating manual rules to identify all 17 classes would be difficult because different classes can have similar visual and spectral characteristics.
- Deep Learning Helps By:
 - Learning patterns directly from the available data
 - Finding useful features automatically
 - Using both image information and spectral information
 - Making predictions for unseen image patches
 - Reducing the need for manually designed rules

WHY U-NET

- We need a model that can do two things:
 - Understand what is present in the image
 - Know exactly where those features are located
- U-Net is well suited for this because it is designed specifically for pixel-level image segmentation.
- Main Advantages of U-Net
 - It can learn important features from the input image.
 - It can preserve useful spatial information.
 - It can produce a prediction for every pixel.
 - It uses skip connections to retain fine details.
 - It works well even when the amount of training data is limited.

HOW DOES U-NET WORK

- U-Net works in three main stages:
 1. Encoder — Understand the image
 2. Bottleneck — Capture important information
 3. Decoder — Create the segmentation map
- The input image first goes through the encoder, where the model learns useful patterns. The decoder then uses this information to produce the final pixel-by-pixel prediction.



U-NET ARCHITECTURE

U-Net has two main paths:

- **Contracting Path — Encoder**

- The left side of the network.
- Gradually reduces the image size.
- Learns important patterns and features from the image.
- As the image becomes smaller, the network learns more complex information.

- **Expanding Path — Decoder**

- The right side of the network.
- Gradually increases the image size again.
- Uses the learned features to create the segmentation result.
- Finally produces a class prediction for every pixel.

U-NET ARCHITECTURE

- **Skip Connections**

- Connect the encoder to the decoder.
- Pass useful details directly from the encoder to the decoder.
- Help the network preserve the location and boundaries of objects.

ENCODER: UNDERSTANDING THE IMAGE

- The encoder is the left side of the U-Net architecture.
- Its main job is to look at the input image and learn important features.
- At the beginning, the network looks at simple information such as:
 - Edges
 - Patterns
 - Textures
 - Local variations
- As we move deeper into the encoder, it learns more complex and meaningful features.

ENCODER: UNDERSTANDING THE IMAGE

- The encoder repeatedly applies:
 - Convolution → Batch Normalization → ReLU → Convolution → Batch Normalization → ReLU → Max Pooling
- The pooling operation reduces the spatial size of the feature maps.
- At the same time, the network can learn increasingly complex features.

ENCODER: UNDERSTANDING THE IMAGE

1. Convolution

- The convolution layer looks at small regions of the input and detects useful patterns.
- At the beginning, these can be simple patterns such as:
 1. Edges
 2. Lines
 3. Textures
 4. Small shapes
- As we go deeper into the network, the convolution layers learn more complex patterns.

ENCODER: UNDERSTANDING THE IMAGE

2. Batch Normalization

- After convolution, **Batch Normalization** is applied.
- Its purpose is to keep the values flowing through the network more stable, which helps the model train effectively.

3. ReLU Activation

- Next, the ReLU activation function is applied.
- It introduces non-linearity into the network, allowing the model to learn complex relationships rather than only simple linear patterns.`

ENCODER: UNDERSTANDING THE IMAGE

4. **Second Convolution Block**

- The second convolution extracts additional features from the information produced by the first convolution.

5. **Max Pooling**

- Finally, Max Pooling reduces the spatial size of the feature map.

BOTTLENECK: THE DEEPEST PART OF U-NET

What is the Bottleneck?

- The **bottleneck** is the deepest part of the U-Net, located between the encoder and decoder.
- By the time the input reaches this stage:
 - The spatial dimensions have been reduced.
 - The network has extracted important features from the image.
 - The feature representation contains more high-level information.
- In our implementation, the encoder reduces the spatial representation to approximately: $128 \times 8 \times 8$

DECODER: RECONSTRUCTING THE SEGMENTATION MAP

What Does the Decoder Do?

- The decoder is the right side of the U-Net.
- Its main job is to take the compact features learned by the encoder and gradually restore the spatial dimensions to produce the final segmentation map.

How Does It Work?

- The decoder mainly performs:
Upsampling → Feature Combination → Convolution → Feature Refinement

SKIP CONNECTIONS: PRESERVING IMPORTANT DETAILS

Why Are Skip Connections Needed?

- When the encoder reduces the image size using pooling, some fine spatial details can be lost.
- For segmentation, these details are important because the model needs to know exactly where each class is located.
- U-Net solves this using skip connections.

• How They Work

- Skip connections directly transfer feature information from the encoder to the corresponding stage of the decoder.
- The decoder combines:

Information from the encoder + Information from the decode

PREPARING THE IMAGE FOR U-NET

From Original Image to U-Net Input

- Our original hyperspectral image is too large to directly use as a single small input to the U-Net.
- Therefore, we divide the image into smaller **32×32 patches**.

Why Do We Use Patches?

- Using smaller patches makes the data easier for the model to process and allows the model to learn from different regions of the image.
- Each patch contains:
 - **200 spectral bands**
 - **32×32 spatial pixels**
- The corresponding ground-truth patch has:
 - **32×32**

PREPARING THE IMAGE FOR U-NET

Deploy ⋮

Image Patches

Total Patches

25

Patch Height

32

Patch Width

32

Image patches shape: `torch.Size([25, 200, 32, 32])`

Mask patches shape: `torch.Size([25, 32, 32])`

TRAINING, VALIDATION AND TESTING

How Do We Train and Evaluate the Model?

- After dividing the image into 25 patches, we separate them into three groups.

Purpose	Patches
Training Dataset	17
Validation Dataset	4
Testing Dataset	4
Total	25

TRAINING, VALIDATION AND TESTING

4 Dataset Split [↗](#)

Training

17

Validation

4

Testing

4

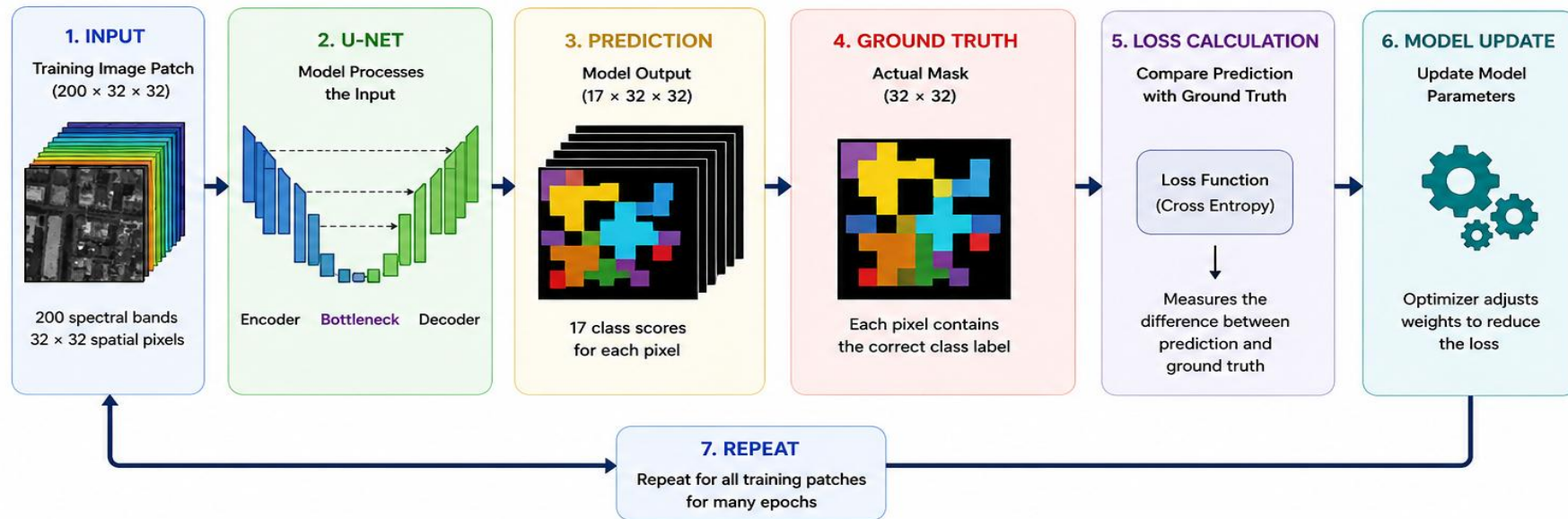
TRAINING PROCESS

- The U-Net learns by comparing its prediction with the correct ground-truth mask.
- The process is repeated for many training cycles called **epochs**.

TRAINING PROCESS

TRAINING WORKFLOW

The model learns by comparing its prediction with the ground truth and updating its parameters to reduce the error.



- 1 Input patch is taken from the training set.
- 2 U-Net extracts features and produces prediction.
- 3 Model predicts 17 class scores for each pixel.
- 4 Ground truth provides the correct class for each pixel.
- 5 Loss is calculated to measure the error.
- 6 Model parameters are updated to reduce the error.
- 7 Process repeats for many epochs until convergence.

KEY INFORMATION

- Input Patch Size : 200 × 32 × 32
- Ground Truth Size : 32 × 32
- Output (Prediction) : 17 × 32 × 32
- Goal : Minimize loss and improve accuracy

LOSS FUNCTION

Cross Entropy Loss is used because this is a multi-class pixel-wise classification task.

OPTIMIZER

Adam Optimizer is used to update the model parameters efficiently.

TRAINING PROCESS

What Happens During Training?

1. Input

- A training patch of size: $200 \times 32 \times 32$ is given to the U-Net.

2. Prediction

- The U-Net predicts a class for every pixel.

3. Comparison

- The prediction is compared with the correct ground-truth mask.

4. Loss Calculation

- The difference between the prediction and the correct answer is measured using a **loss function**.

5. Model Update

- The model adjusts its internal parameters so that its future predictions can become better.

TRAINING RESULTS

Deploy ⋮

5 U-Net Training

Click the button below to train the U-Net using the same training function used by your original project.

Start Training

Training completed successfully!

6 Training Graphs ↔

Training and Validation Results

Training Accuracy

96.04%

Training Loss

0.0000

Validation Accuracy

68.24%

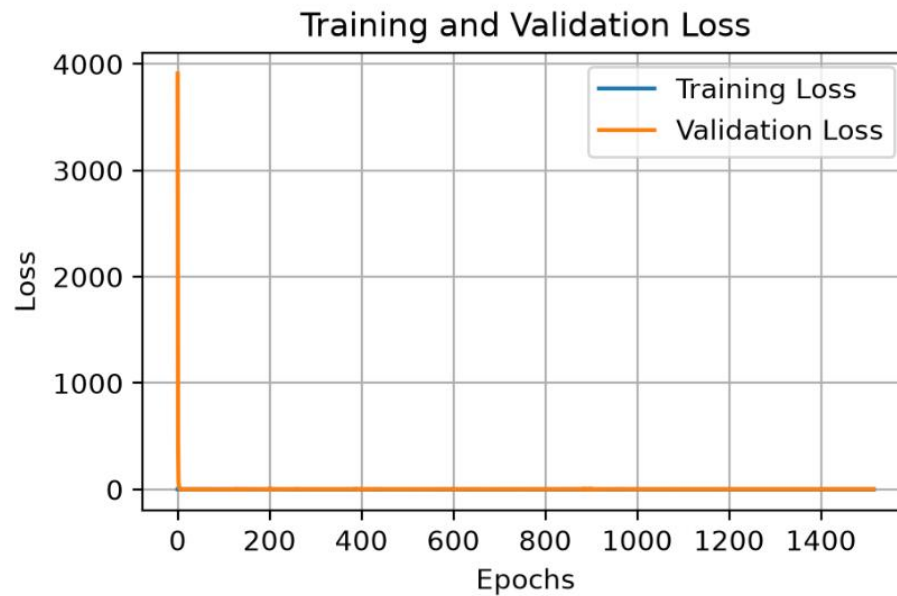
Validation Loss

3.1390

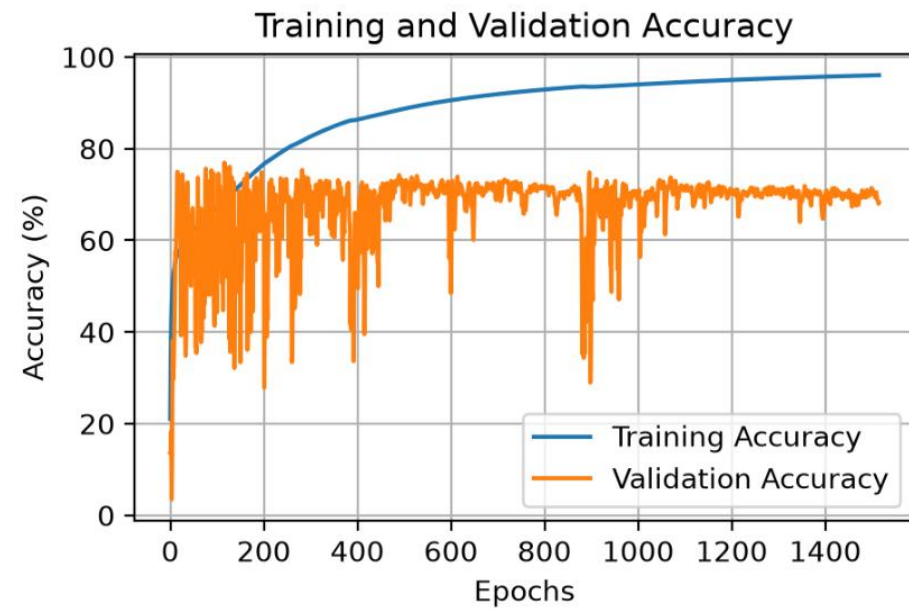
TRAINING RESULTS

Deploy ⋮

Loss Curve



Accuracy Curve



VALIDATION AND PREDICTION

How Does the Trained Model Perform on Unseen Data?

- After training, we test the model on image patches that were **not used to train the model**.
- For validation, the model compares its predicted mask with the actual ground-truth mask.

VALIDATION RESULT

Deploy ⋮

7 Validation Prediction

Run Validation Prediction

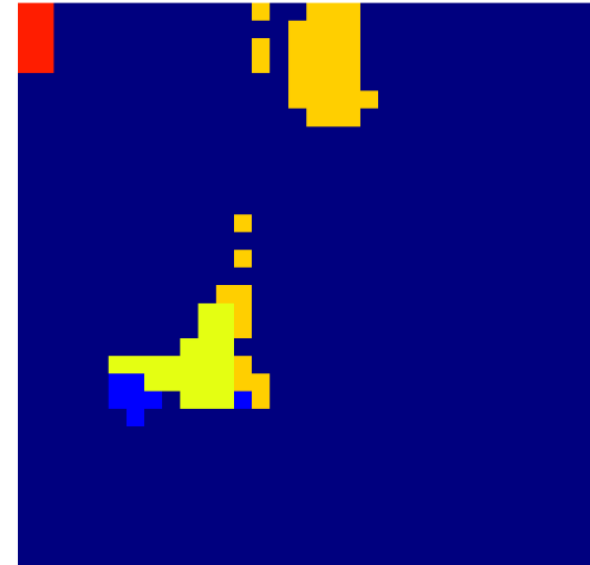


Validation Image - Band 0

True Target Mask



AI Validation Prediction



TESTING RESULTS

Final Evaluation on Unseen Data

- After training and validation, the model is evaluated on the **4 test patches** that were kept separate from the training process.

TESTING RESULTS

Deploy ⋮

8 Testing

Run Testing

Test Loss

3.4813

Test Accuracy

65.21%



Test Image - Band 0

True Target Mask



U-Net Prediction



FULL IMAGE RECONSTRUCTION

Reconstructing the Complete Segmentation Map

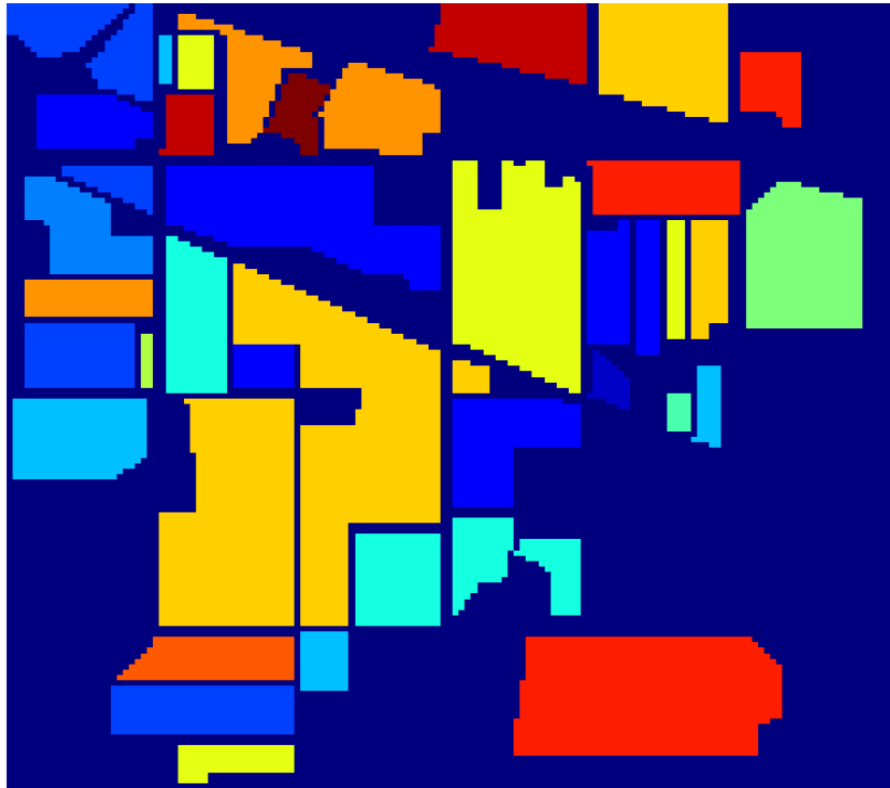
- Until now, the U-Net has been working with individual **32×32 patches**.
- To obtain the segmentation result for the complete image, we combine the predictions from all **25 patches** and place them back in their original positions.

FULL IMAGE RECONSTRUCTION RESULT

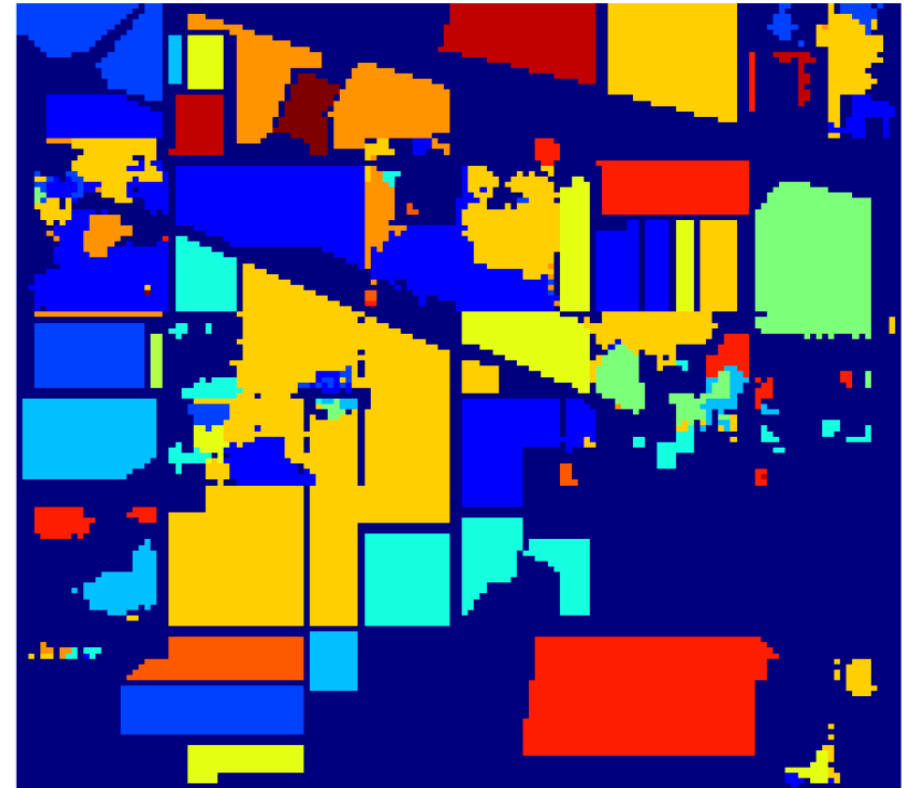
Full Image Reconstruction Accuracy

86.44%

Complete Ground Truth Mask



Complete U-Net Predicted Mask





THANK YOU

Questions & Discussion

U-Net for Image Segmentation used items are :-

Indian Pines Hyperspectral Dataset

Python • PyTorch • Streamlit